

Assignment 4

Enhancing Steel Defect Detection Using Segmentation-Based Preprocessing

Task:

You have to build a deep learning pipeline to classify steel surface images into the following four defect categories:

1. **Crazing**
2. **Patch**
3. **Pitted Surface**
4. **Scratch**

Begin by implementing a standard CNN classifier. Then, explore whether **simple segmentation-based preprocessing**, using only traditional computer vision techniques, can improve the classification performance. **No ground-truth segmentation masks are provided.**

Dataset:

- The dataset contains classification labels for the images.
 - There is a **significant class imbalance**, with the "Scratch" category forming a large portion of the dataset.
 - You are required to apply techniques to handle this imbalance appropriately (e.g., class weighting, oversampling, or targeted augmentation).
-

Breakdown:

Part 1: Baseline Multi-Class Classification Using CNN

1. **Preprocessing:**
 - Resize, normalize input images.
 - Analyze and visualize the class distribution.
 - Apply strategies to mitigate class imbalance.
 2. **Model Development:**
 - Implement a CNN-based classifier using TensorFlow or Keras.
 - Evaluate the model using metrics like accuracy, precision, recall, F1-score, and confusion matrix.
-

Part 2: Segmentation-Based Preprocessing for Improved Classification

Use **traditional image processing techniques** to extract probable defect regions. No labeled masks are available, so rely on **unsupervised segmentation** approaches such as:

- **Edge detection** (e.g., Canny, Sobel)
- **Thresholding** (e.g., Otsu's method, adaptive thresholding)
- **Contour detection**
- **Morphological operations** (e.g., dilation, erosion)

Apply the extracted defect regions to improve your classification model by:

- Masking background regions to reduce noise.
- Cropping to defect-focused regions before feeding into the classifier.
- Adding handcrafted features (e.g., size or shape of detected regions) alongside CNN features.

Then, retrain your classifier using the segmentation-informed inputs and compare performance.

Deliverables:

1. **Code (Notebook or Script):**

- Baseline CNN classifier.
- Segmentation-based preprocessing pipeline.
- Comparative evaluation and visualizations.

2. **Report (pages):**

- Description of the dataset and **class imbalance approach**.
- Explanation of segmentation techniques used.
- Comparison of **model performance** (baseline vs segmentation-based).
- Sample segmented images and classification results.
- **Day-wise efforts** you made and **challenges** you faced.

3. **README File:**

- Instructions to setup and run the code.
-

Important Notes:

- Do **not** use any pre-trained segmentation models or deep learning-based methods.
- The focus is on using **simple, explainable image processing** to enhance classification when segmentation masks are unavailable.